

Functions in C

```
function_type function_name(parameter list)
{
    local variable declaration;
    executable statement1;
    executable statement2;
    . . . . .
    . . . . .
    return statement;
}
```

```
float mul (float x, float y)
{
    float result; /* local variable */
    result = x * y; /* computes the product */
    return result; /* returns the result */
}
```

```
void sum (int a, int b)
{
    printf ("sum = %s", a + b); /* no local variables */
    return; /* optional */
}
```

```
void display (void)
{ /* no local variables */
    printf ("No type, no parameters");
    /* no return statement */
}
```

```
main( )
{
    int y;
    y = mul(10,5);
    printf("%d\n", y);
}
```

/ Function call */*

```
main ( )
{
    int y;
    y = mul(10,5); /* call*/
    ....
}

int mul(int x,int y)
{
    int p; /* local variable*/
    p = x * y; /* x = 10, y = 5*/
    return (p);
}
```

function defⁿ

Function Declaration

- Function declaration also known as function prototype
- Function type (return type).
- Function name.
- Parameter list.
- Terminating semicolon.

```
[int mul (int m, int n); /* Function prototype */
```

→ int mul (int, int); ←

→ main ()

{

int y = mul (10, 5)

}

first time

int mul (int x, int y)

{ return x*y

}

~~No error~~

int mul (int x, int y)

{

return x*y

}

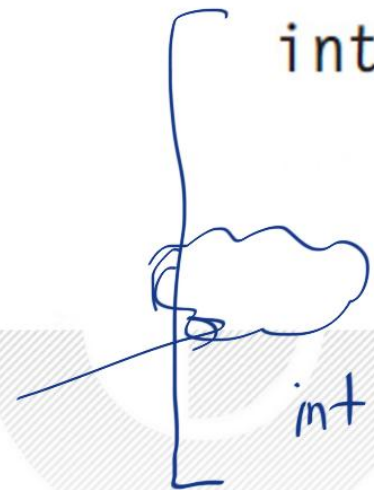
→ main ()

{

int y = mul (10, 5)

}

Equally acceptable forms of declaration of **mul** function are:



```
int mul (int, int); ✓  
mul (int a, int b); ✓  
mul (int, int); ✓  
int mul (int a, int b); ✓
```

short x;



short int x;

Using a function

```
#include <stdio.h>

int fun(int x, int y); //function declaration

main()
{
    int a;
    a = fun(2, 3);
    printf("%d", a);
}

int fun(int x, int y) //function definition
{
    return x+y;
}
```

Using a function

```
#include <stdio.h>

int fun(int x, int y); //function declaration

main()
{
    int a;
    a = fun(2, 3);
    printf("%d", a);
}
```

NO compilation error
but there will be run time error

Will that work ?



yes

```
{ #include <stdio.h>
  main()
  {
    int a;
    a=fun(2,3);
    printf("%d", a);
  }
```

[fun is a function which has
return type int.]

```
int fun(int x, int y){
    return x+y;
}
```

Will that work ? (Solution)

```
[ -  
#include <stdio.h>  
main()  
{  
    int a;  
    a=fun(2,3);  
    printf("%d", a);  
}  
  
int fun(int x, int y)  
{  
    return x+y;  
}
```

```
$gcc -std=c99 -o main *.c  
main.c:6:1: warning: return type defaults to 'int' [-Wimplicit-int]  
main()  
^~~~  
main.c: In function 'main':  
main.c:9:9: warning: implicit declaration of function 'fun' [-Wimplicit  
-function-declaration]  
    a = fun(2, 3); /* The compiler has not seen the declaration. */  
               ^~~  
$main  
5
```

Will that work ? → NO

Compiler
assumption

int as
return type

```
#include <stdio.h>

main()
{
    char a;
    a = fun(2, 3); /* The compiler has not seen the declaration. */
    printf("%c", a);
}

char fun(int x, int y)
{
    return x+y;
}
```

Will that work ?

No

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    float a = foo();
```

```
}
```

```
float foo()
```

```
{
```

```
    return 3.5;
```

```
}
```

Compiler assumes
int as return type

but later found

float

GATE CSE 2005

Consider the following C program:

```
X [double foo (double);           /* Line 1 */
int main() {
    double da, db;
    //input da
    db = foo(da);
}
double foo (double a) {
    return a;
}
```

Handwritten annotations:

- An arrow points from the `double` in the function signature to the text "compiler assume int".
- An arrow points from the `double` in the function definition to the text "later found double".

The above code compiled without any error or warning. If Line 1 is deleted, the above code will show:

- A. no compile warning or error
- B. some compiler-warnings not leading to unintended results
- C. some compiler-warnings due to type-mismatch eventually leading to unintended results

✓ D. compiler errors

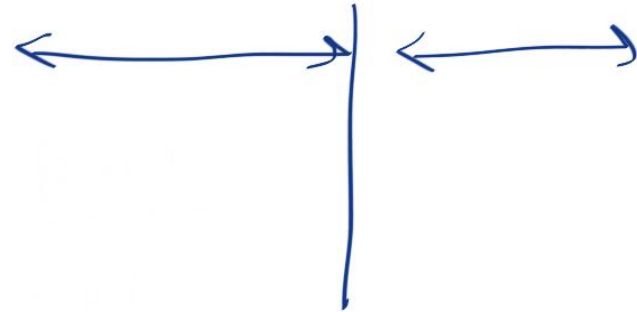
Activation record



Compiler design Subject



for every fn call



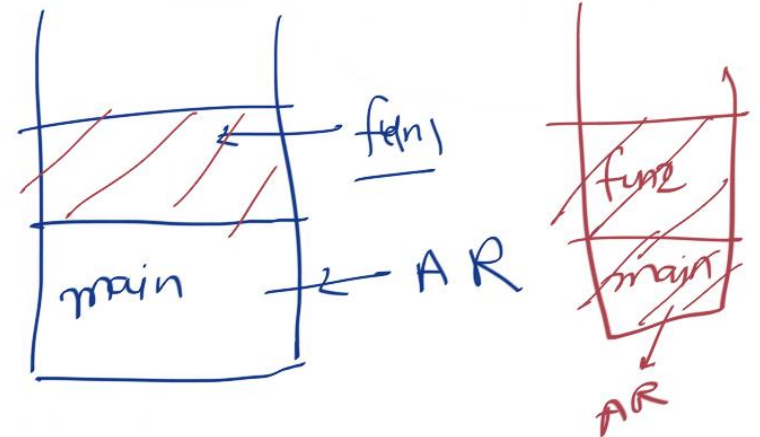
main ()

{

fun1 () ;

fun2 () ;

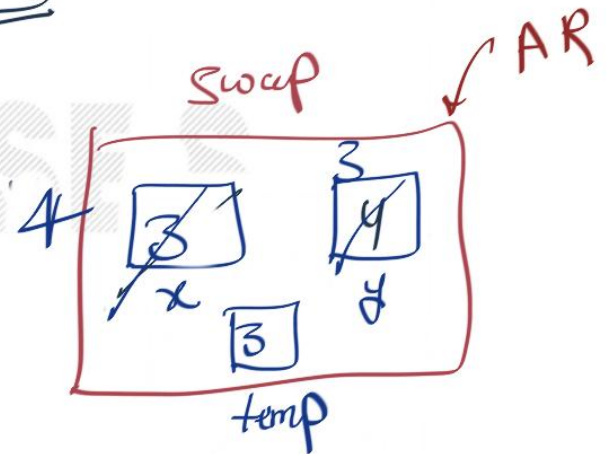
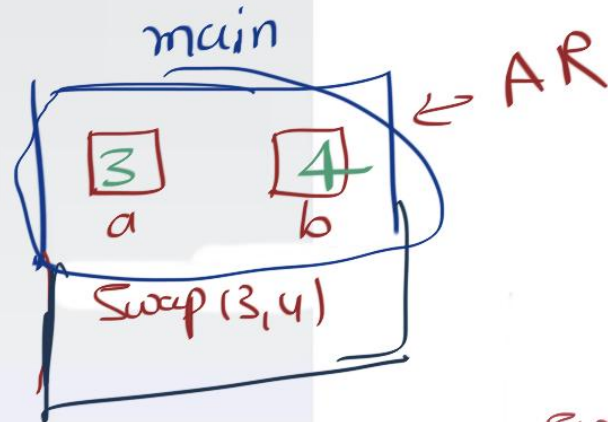
}



```
void swap(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
}
```

```
int main(void) {
    int a = 3;
    int b = 4;
    swap(a, b);
    printf("a = %d, b = %d\n", a, b);
}
```

3, 4



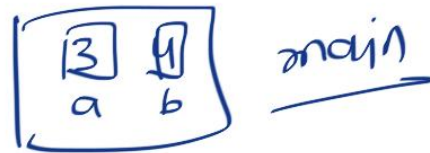

```
main ( )
```

```
{ int a = 3, b = 4
```

```
    fun (a, b)
```

```
    print (a, b)
```

```
    }  
    }  
    3, 4
```



```
fun ( int x, int y)
```

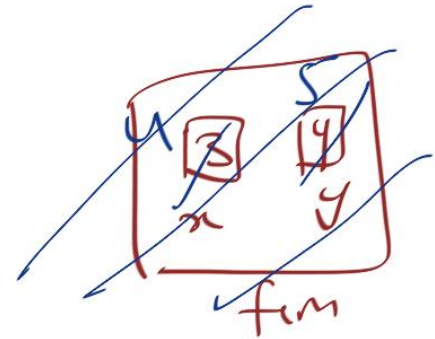
```
{
```

```
    x = x + 1
```

```
    y = y + 1
```

```
    return ;
```

```
}
```




```
main ( )
```

```
{ int a = 2, b = 3;
```

```
  fun (a, b)
```

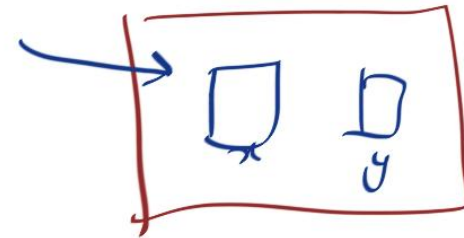
```
}
```

```
fun (int x, int y)
```

```
{
```

```
}
```

all local variables

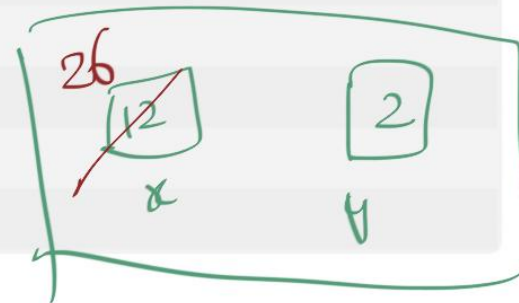
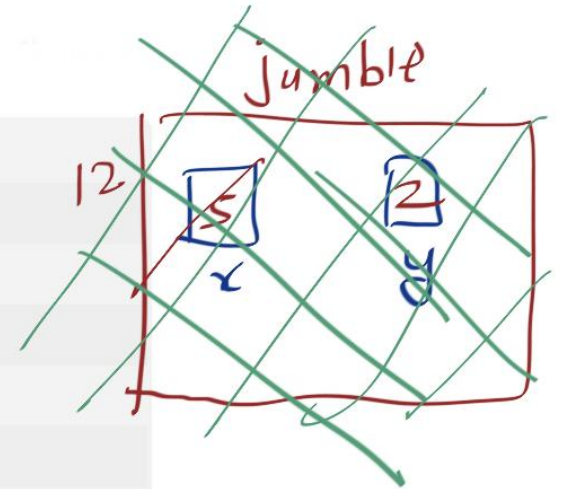
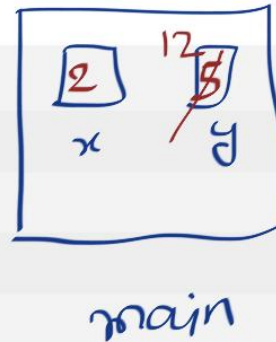


fun

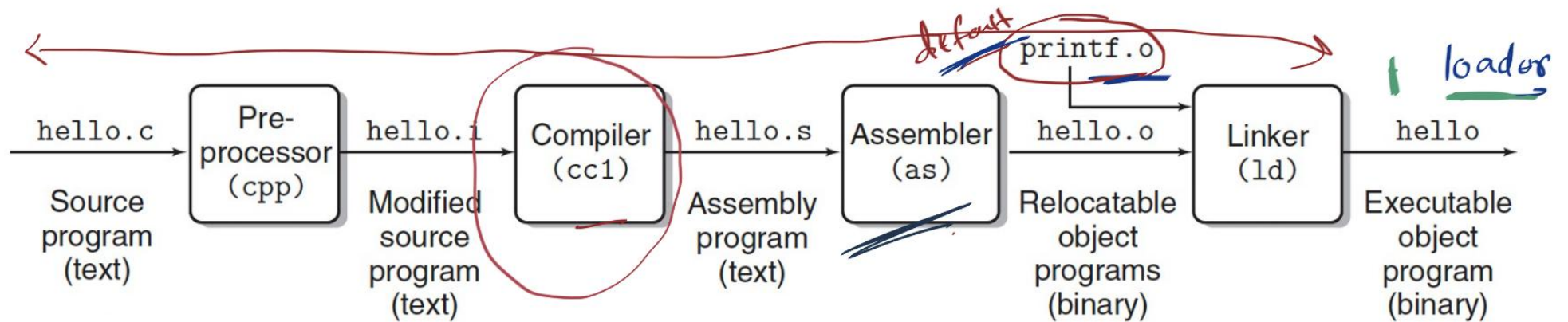
GATE CSE 2019 Question 18

Consider the following C program :

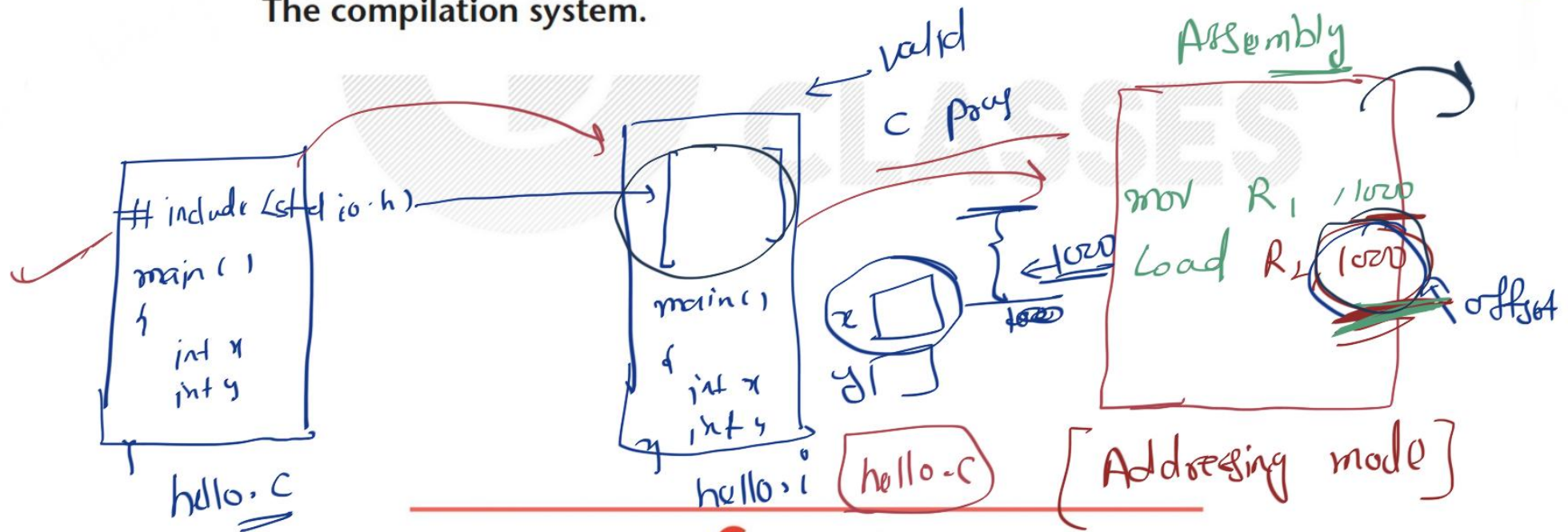
```
#include<stdio.h>
int jumble(int x, int y){
    x = 2*x+y;
    return x;
}
int main(){
    int x=2, y=5;
    y=jumble(y,x);
    x=jumble(y,x);
    printf("%d \n",x);
    return 0;
}
```

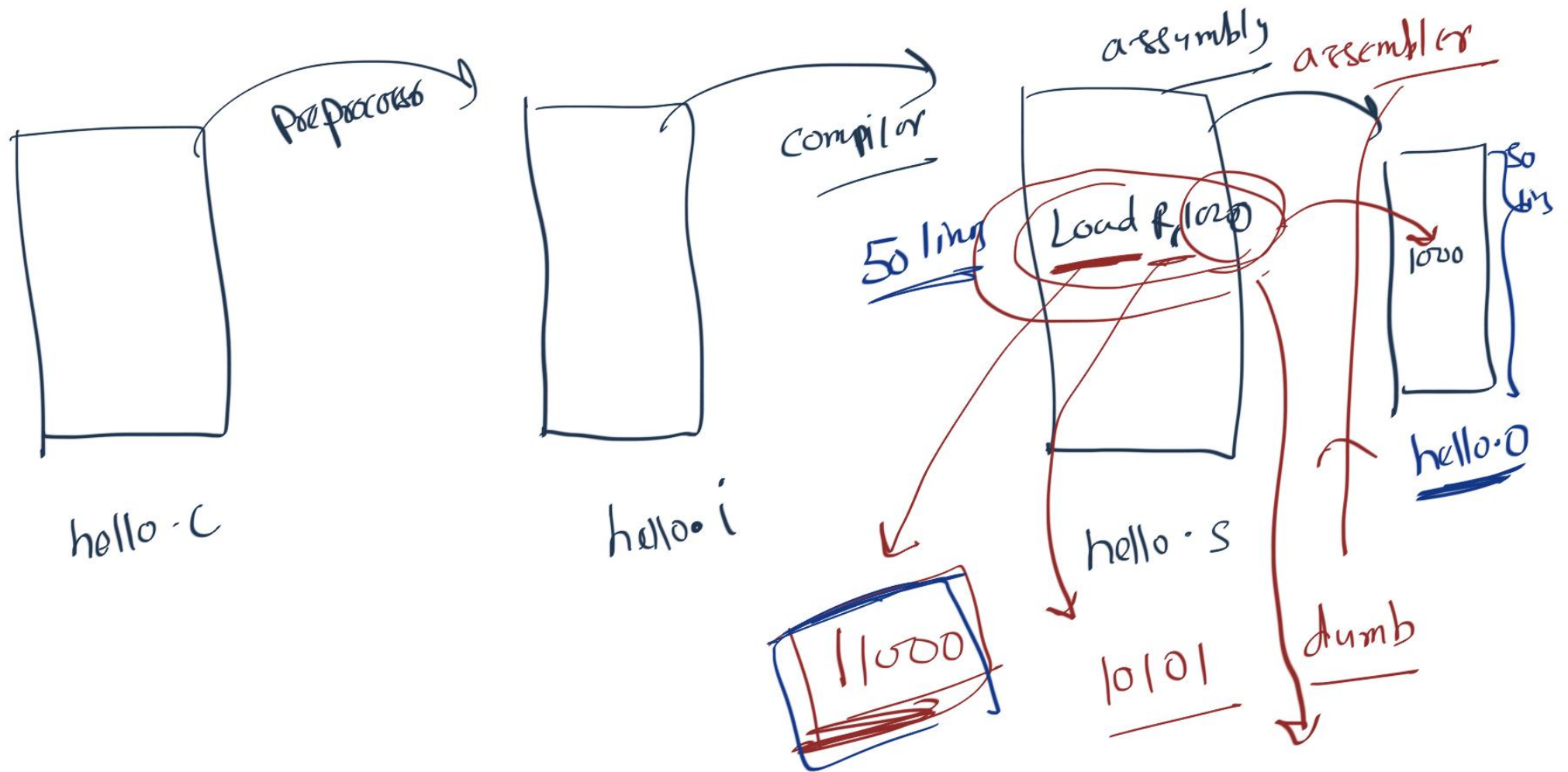


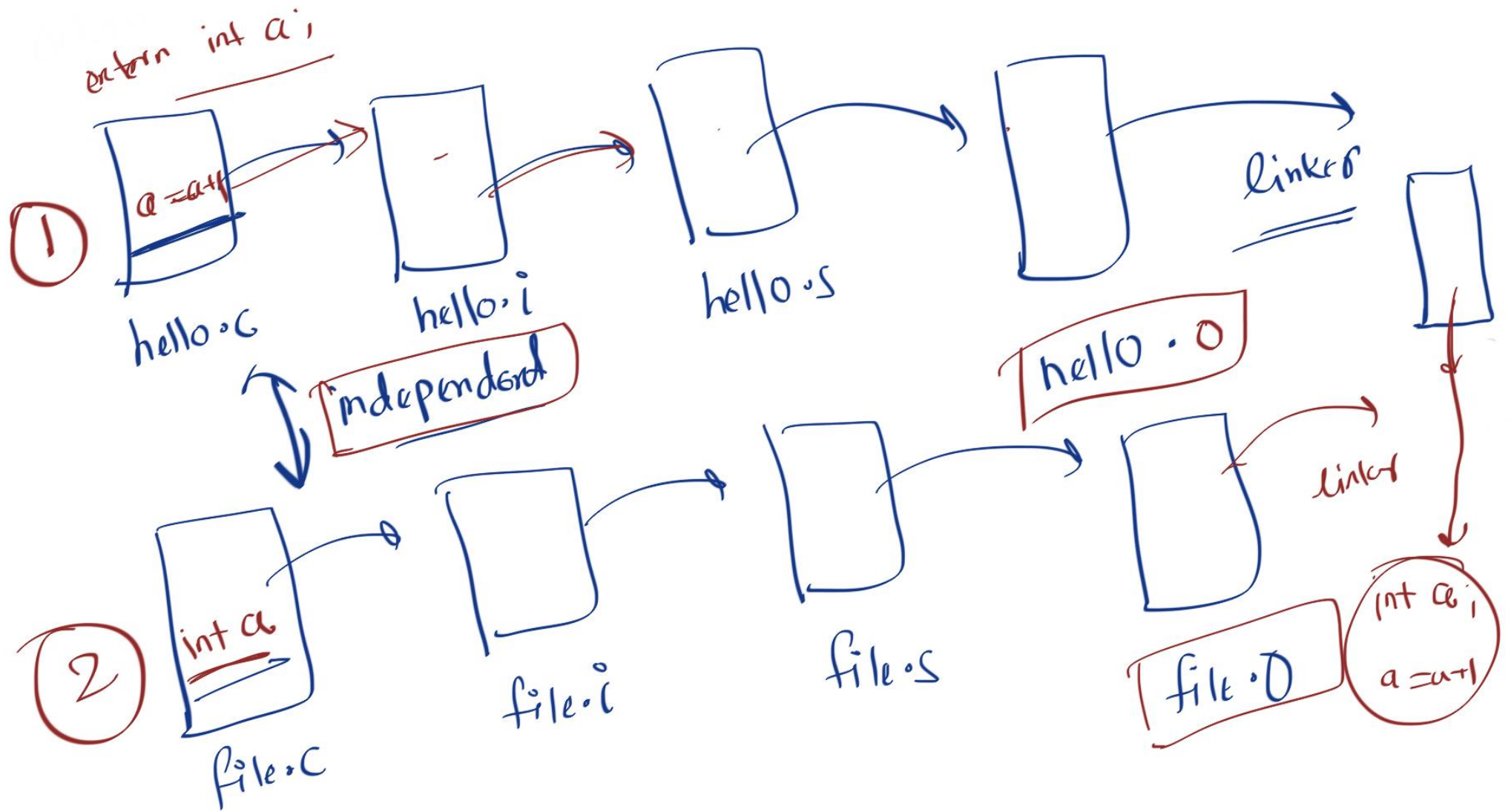
The value printed by the program is _____.



The compilation system.







prototype

hello.c

indep.

hello.o

hello.c

```
int mul(int, int);  
main {  
    y = mul(2, 3)  
}
```

int mul(int a, int b)
{ return a * b;
}

define

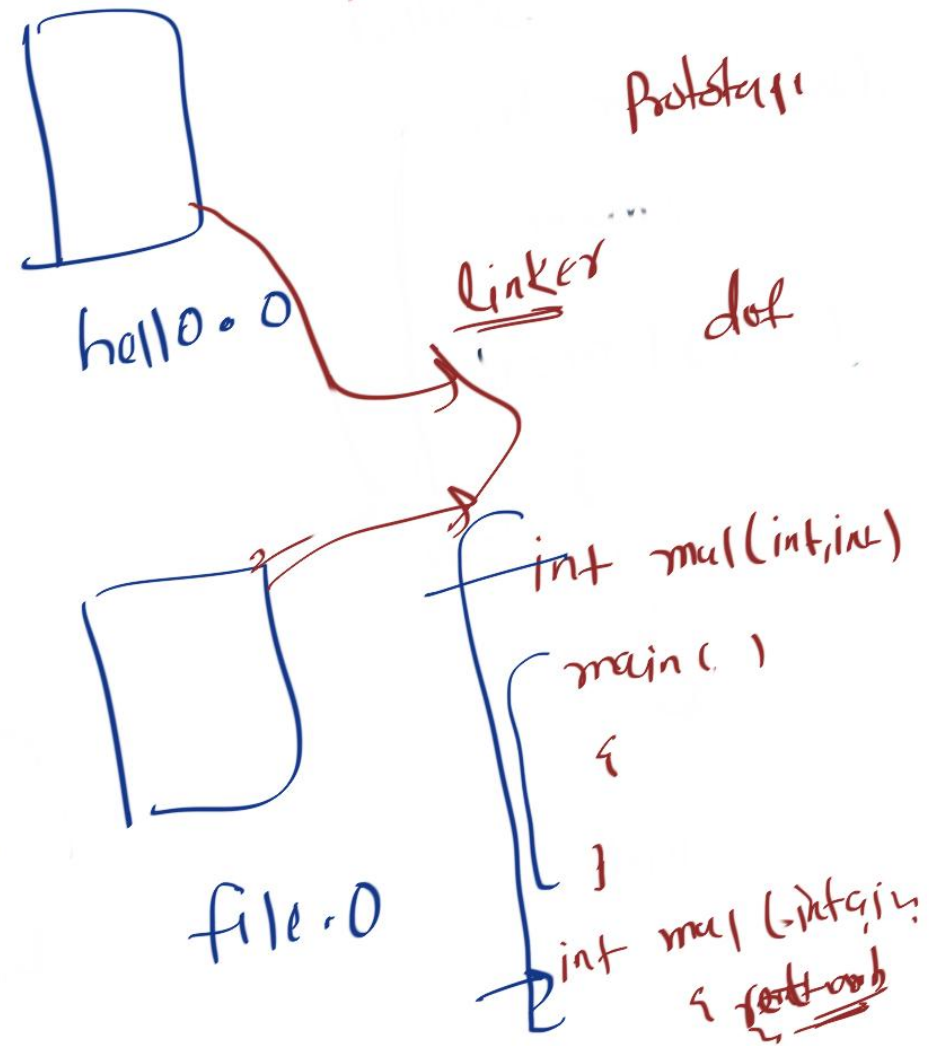
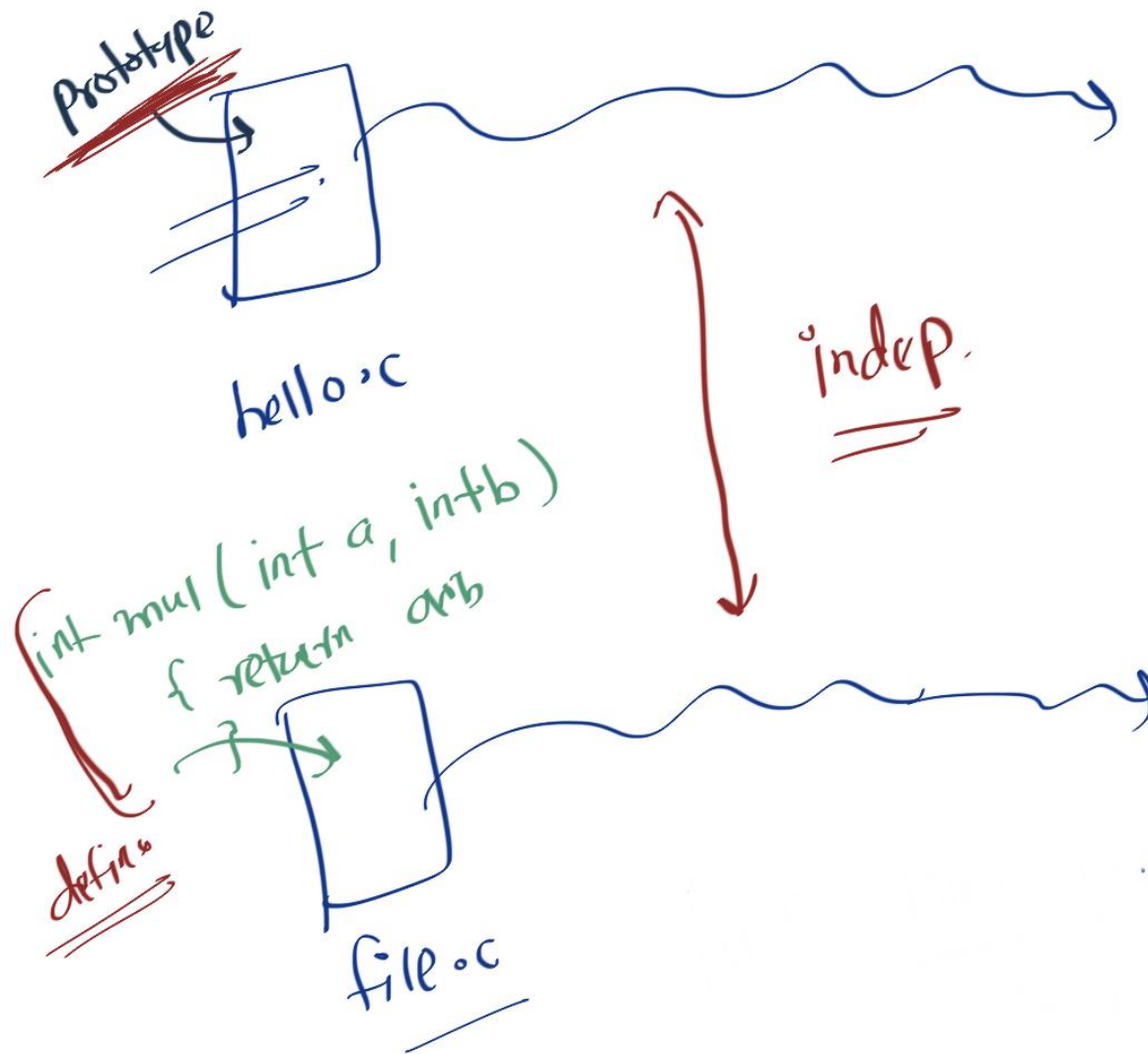
file.c

gcc file.c hello.c

a.exe

file.o

hello.o



extern int a;

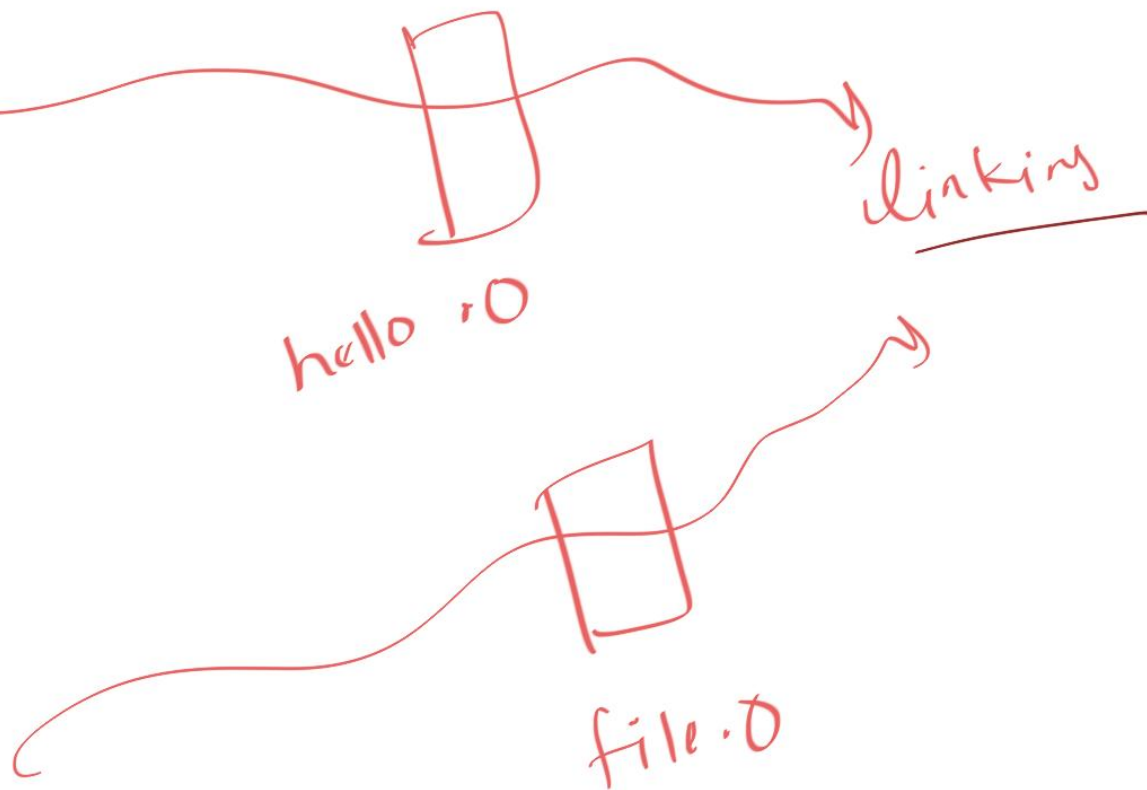
a=a+1

hello.o

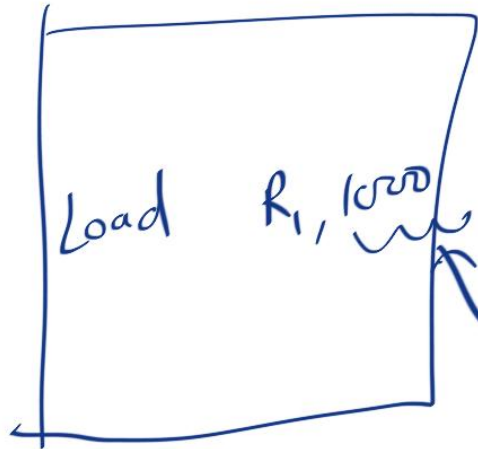
linking

int a;

file.o



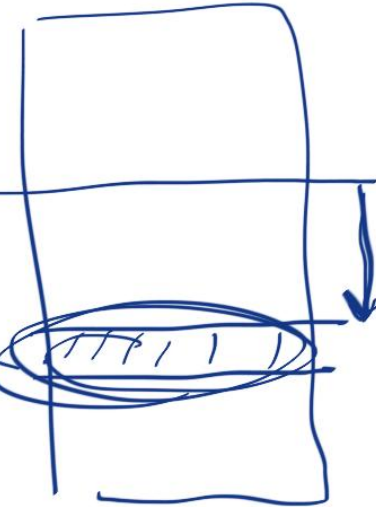
base
address



effective

decimal 0x

0000ffff



1000

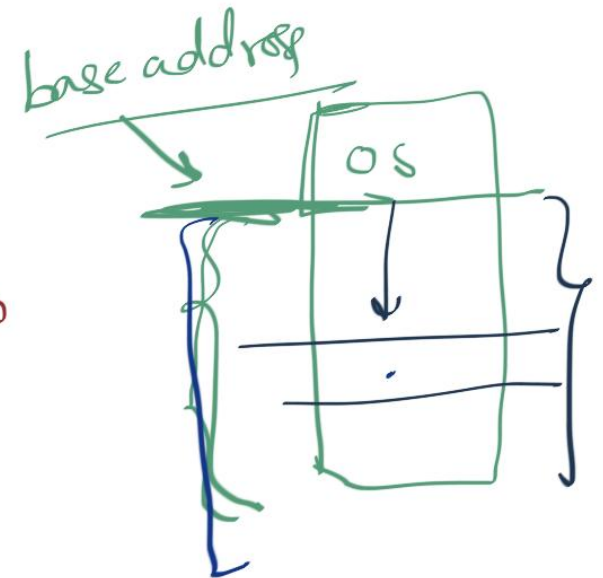
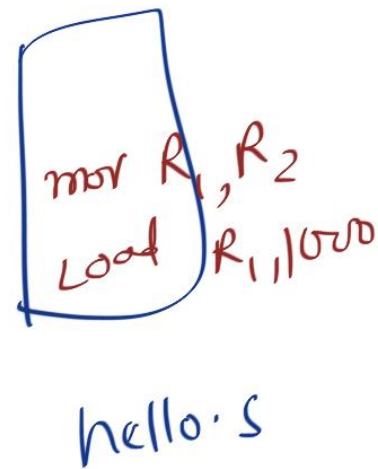
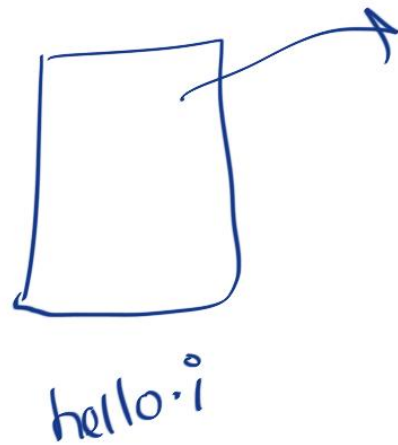
hex
0000ffff + 1000

effective

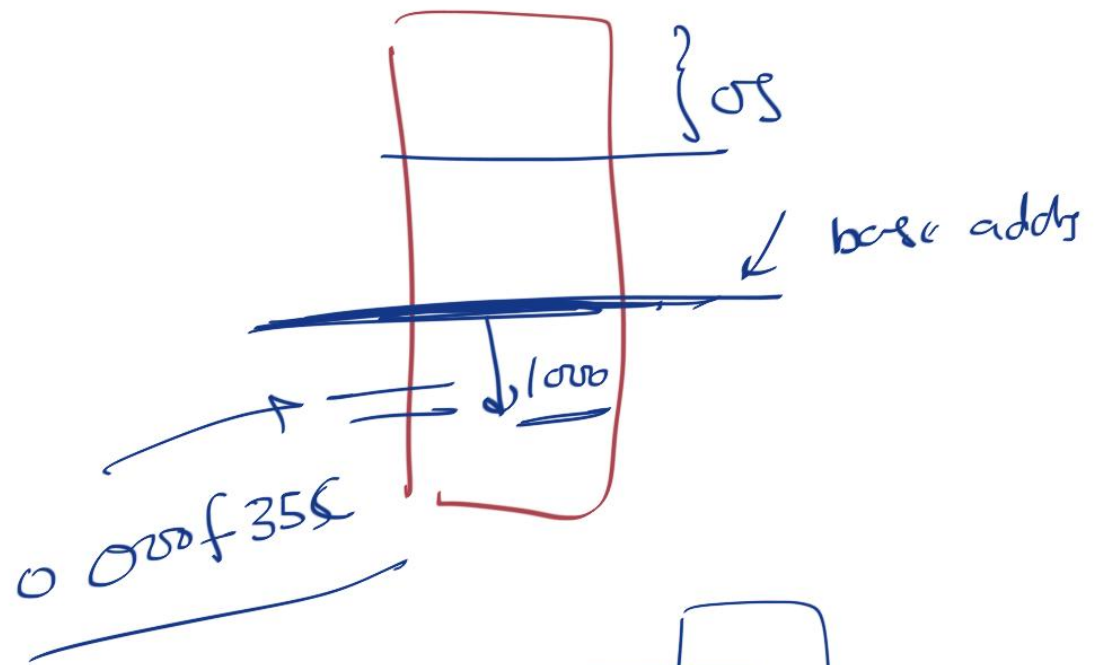
R1
address

Virtual memory : — “Most successful idea in computer science”

complete memory is available for 1 program



R1, 1000



#include <stdio.h>

→ main {

→ {

→

}

hello

printf ("hello")

↪ return type of int
(assumption)

int

hello = 0

linking

printf.o

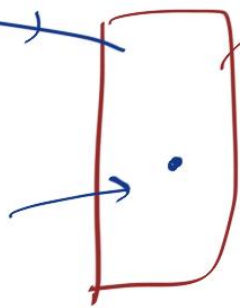
stdio.h

Prototype

int printf (char* s, arg)

float

~~[int printf &]~~



hello.c



hello.o



scanf.o

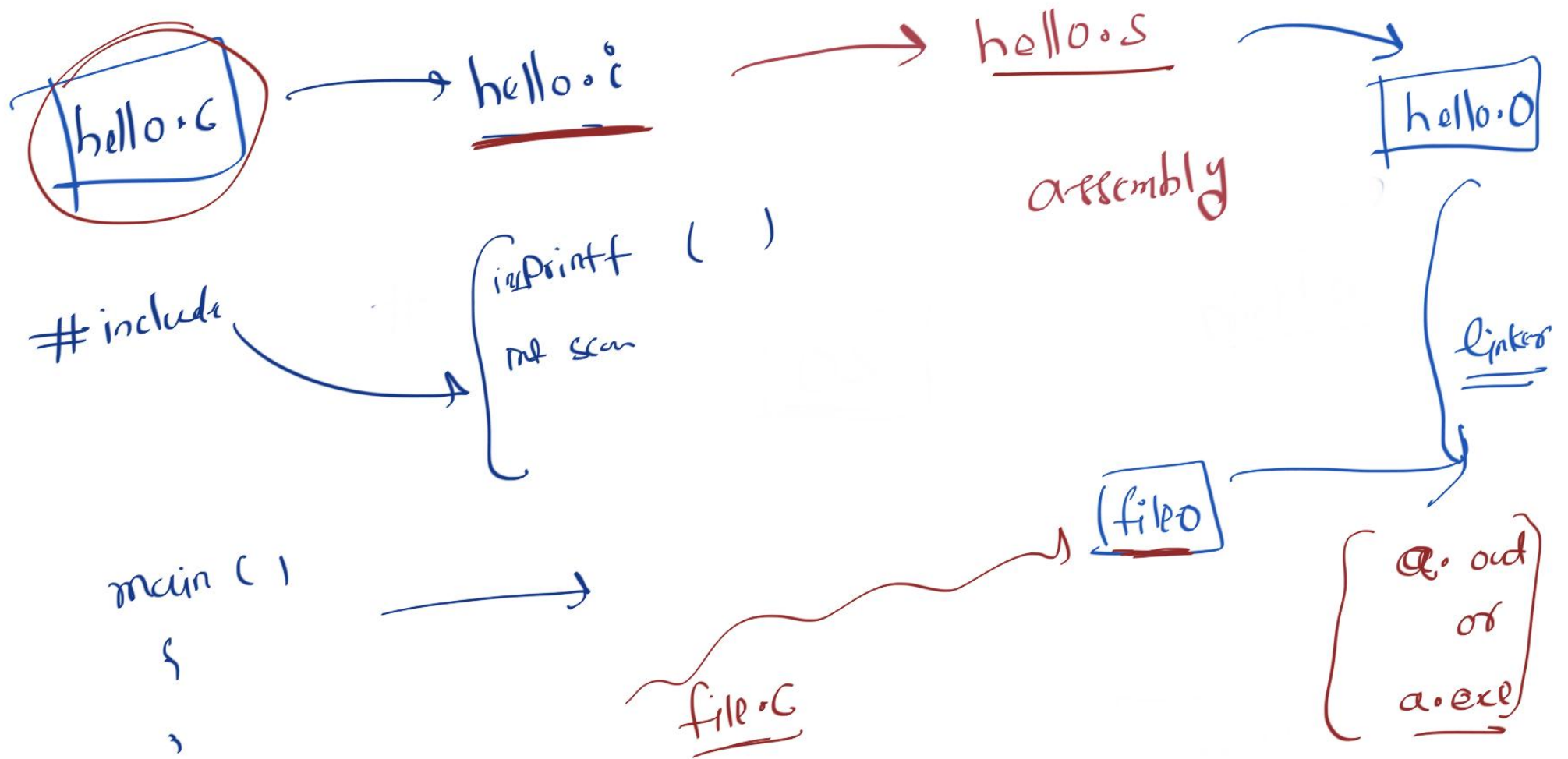


printf.o

a.out

or

a.exe



```
#include < >
```

```
main ( )
```

```
{
```

```
int y = mul (2, 3);
```

```
}
```

linking

hello.o

printf.o

scanf.o

assumed
to have int
as return

working code -

hello.c

```
#include <stdio.h>
```

```
main ( )
```

```
{
```

```
int x = mul(2, 3);
```

```
printf ("%d", x);
```

```
}
```

file.c

```
int mul (int x, int y)
```

```
{
```

```
return x*y
```

```
}
```

gcc hello.c file.c → a.out

"/a.out

6

working code -

hello.c

```
#include <stdio.h>
```

```
main ( )
```

```
{
```

```
    int x = mul(2,3);
```

```
    printf ("%d", x);
```

```
}
```

file.c

```
int mul (int x, int y)
```

```
{
```

```
    return x*y
```

```
}
```

gcc hello.c file.c → a.out
-o a.out

after
linking

#include -

```
int mul (int x, int y)
```

```
{
```

```
    return x*y
```

```
}
```

```
main ( )
```

```
{
```

```
    int x = mul(2,3)
```

```
    Pf(x)
```

```
}
```

File 1

main ()

{

}

File 2

main ()

{

}

Can not link

main ()

{

}

main ()

{

}

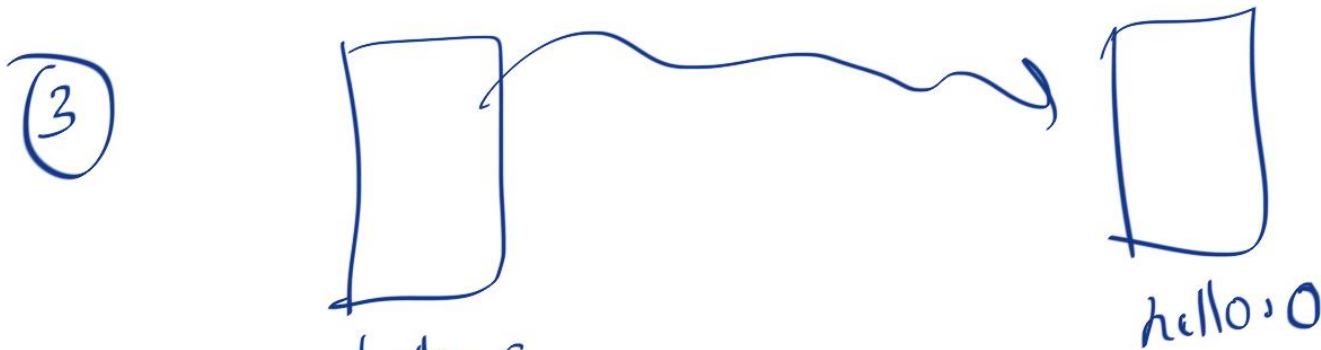
file.c gcc file.c a.out

```
int mul (int x, int y)
{
    return x * y;
}
```

Summary

① default return for function is int

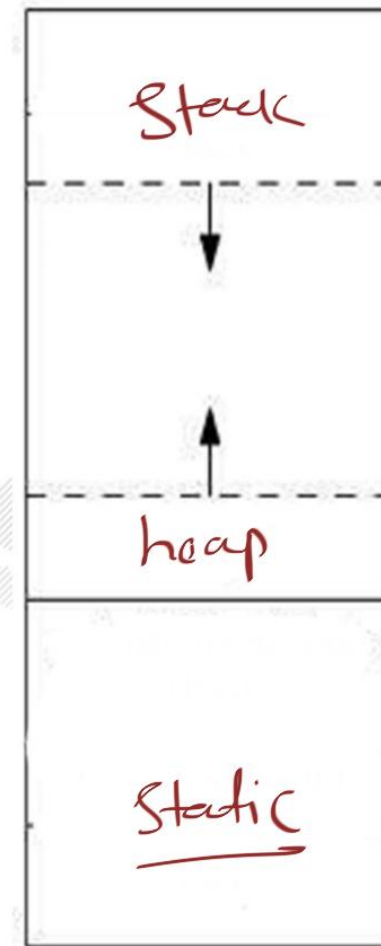
② forgot about vm (sorry for vm details)



④ hello.c
linking

[5-7 class
in OS]

Memory Layout of C program



2 min